

Proyecto Chirp: Diseño y Arquitectura Serverless

Plataforma de microblogging tolerante a escala y de baja latencia en AWS.

STATUS: EN REVISIÓN | REPO: [twitter-core](#) + [cdk](#) | ENV: AWS FREE TIER

El Equipo de Ingeniería



**José Luis
Daza
Sandi**



**Jorge
Carlos
Chávez
Ruiz**



**Javier
Alejandro
Alcoba
Gutiérrez**



**Moisés
Navajas
Bernal**



**Mauricio
David
Gordillo
Herbas**

Desarrollo basado en repositorios acoplados (Frontend, Smithy-Core, AWS CDK).



Alcance y Desafío del MVP

El Problema: Diseñar un sistema capaz de gestionar volúmenes asimétricos de lectura/escritura y generar timelines personalizados sin bloquear la respuesta de la interfaz.

La Condición: Mantener un mantenimiento operativo cercano a cero utilizando exclusivamente servicios gestionados en la nube.

El Alcance: Foco en un conjunto mínimo viable (Crear, Seguir, Interacciones) para validar decisiones clave de modelo de datos, seguridad y latencia estructural.

Contrato de Requerimientos

Requerimientos Funcionales

RF1
Creación y eliminación de chirps
(≤280 caracteres).

RF2
Generación eficiente de timeline
personalizado.

RF3
Sistema de interacciones sociales
(Likes).

RF4
Grafo de red de usuarios
(Followers / Following).

Requerimientos No Funcionales

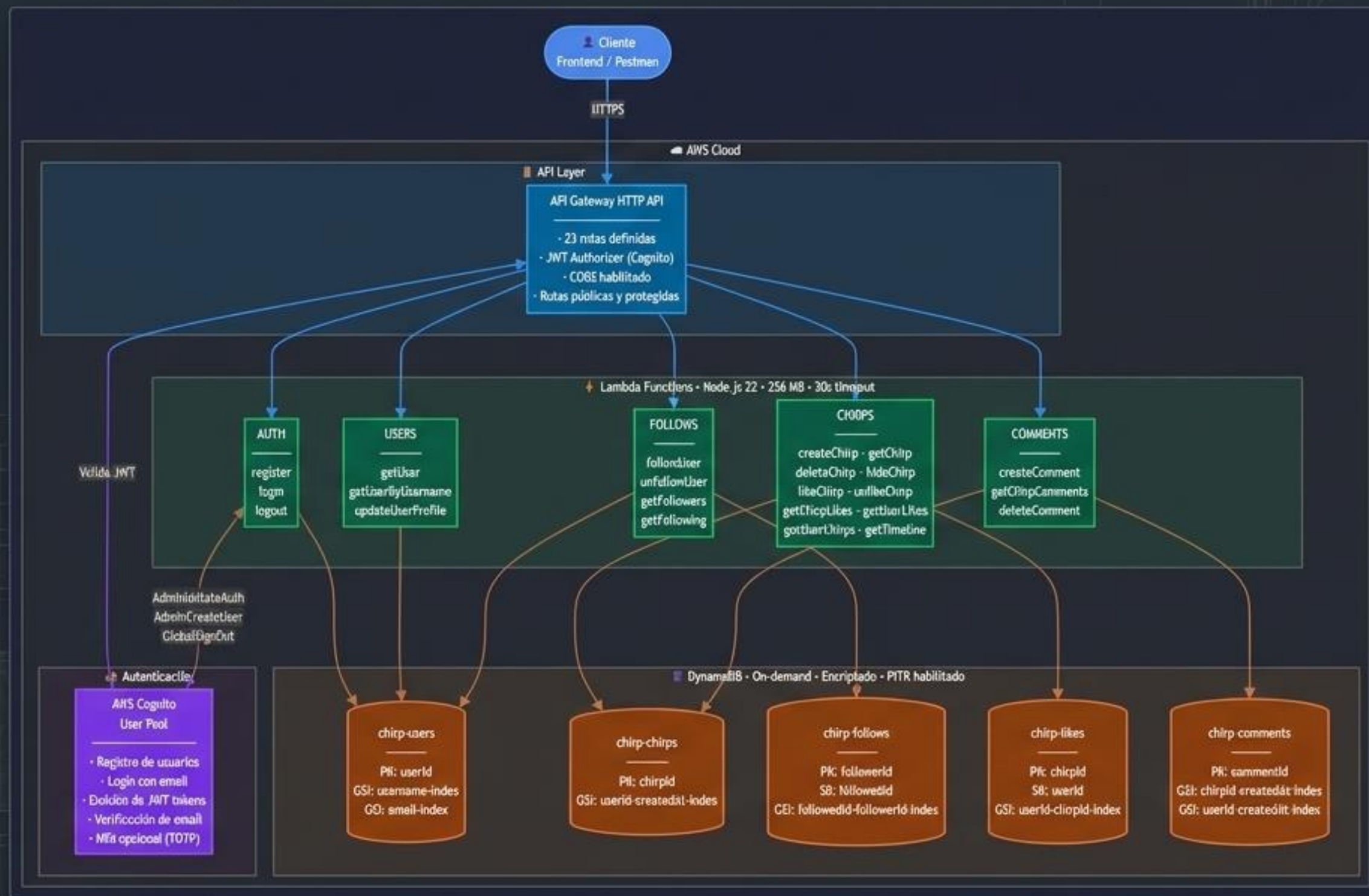
 **LATENCIA:**
Respuesta API garantizada
< 800 ms.

 **CAPACIDAD:**
Soporte base para 500 DAU
(100 concurrentes).

 **DISPONIBILIDAD:**
Diseño tolerante a fallos
locales (Alta disponibilidad).

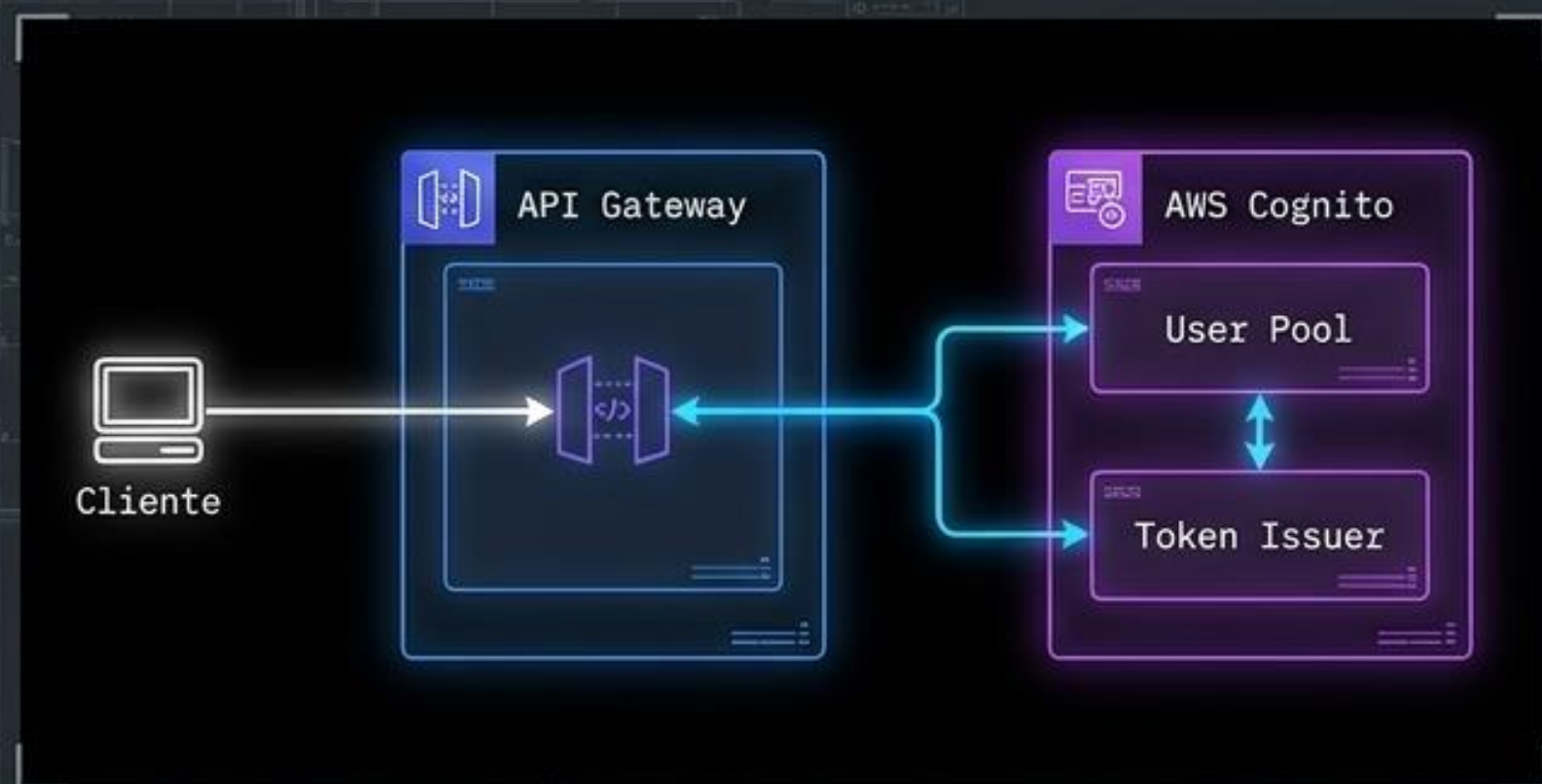
 **SEGURIDAD:**
Intercambio estricto de
identidad vía JWT (OIDC).

Arquitectura Topológica 100% Serverless



El flujo aísla la computación en microservicios independientes, delegando el estado a almacenamiento on-demand y la identidad a un proveedor externo.

Capa Perimetral y Control de Identidad



API Gateway HTTP API: Superficie con 23 rutas definidas y reglas CORS estrictas.

AWS Cognito (User Pool): Gestión de registro, login, verificación de email y emisión de tokens.

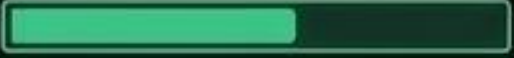
⚠ Decisión Crítica de Diseño

Derivación Inmutable de Identidad.

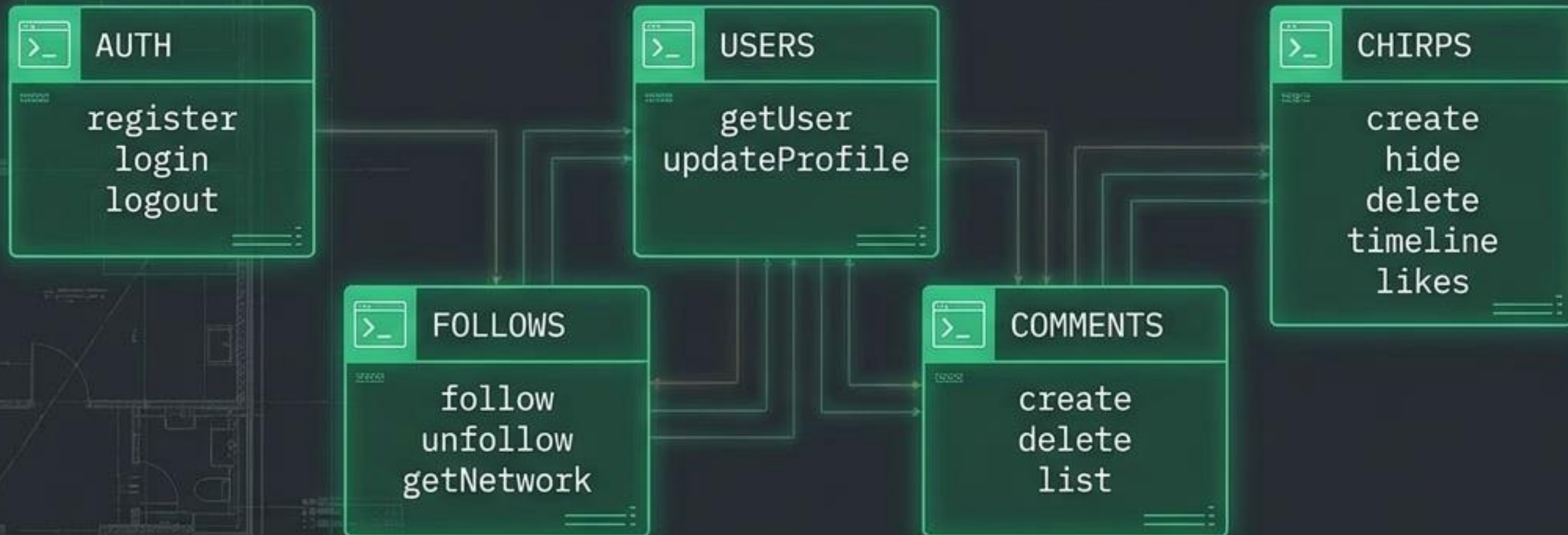
El parámetro `userId` jamás se acepta del payload del cliente. Se extrae de forma exclusiva y en el backend a partir del claim `sub` del token JWT validado por el Authorizer. Esto previene cualquier vulnerabilidad de manipulación de estado.

Capa de Cómputo y Lógica de Negocio

 Node.js 22


 256 MB RAM


 Timeout: 30s

Principio: Despliegue independiente por función. Si el tráfico de lectura de getTimeline sufre picos, no afecta el proceso de register.

Persistencia DynamoDB y Modelado de Datos

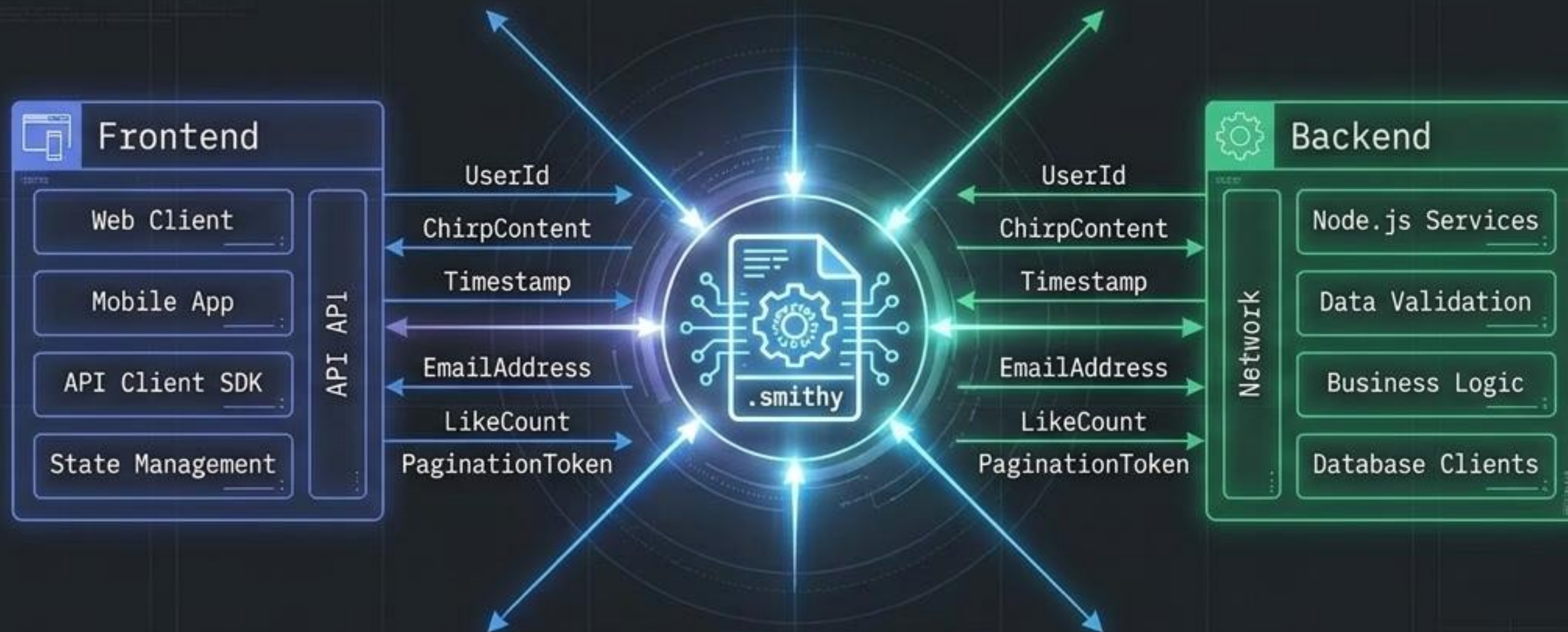
On-demand | Encriptación nativa | PITR habilitado



Modelo de Consistencia Híbrida

Consistencia fuerte para la creación de chirps (integridad esencial), consistencia eventual para interacciones sociales y conteo de likes (rendimiento).

Gobernanza por Contratos: Smithy



El Paradigma

En lugar de documentar el código, el diseño del API dicta el código.

Implementación

1 Única Fuente de Verdad

Los modelos AWS Smithy actúan como la Única Fuente de Verdad.

2 Definición Contractual

Los archivos `.smithy` definen los payloads, tipos (UUID, Email), errores y límites de tamaño (ej. `@length` para chirps).

3 Distribución vía Git Submodules

Utilización de Git Submodules (Twitter-core) para compartir la definición contractual en toda la arquitectura antes de escribir una sola línea de lógica de negocio.

Flujo de Desarrollo (De Smithy al Deploy)



Arquitectura Resolutiva

Al aislar la lógica en microservicios efímeros (**Lambda**) y delegar el estado a estructuras NoSQL indexadas dinámicamente (**DynamoDB**), el sistema cumple las métricas de capacidad (500 DAU, latencia < 800ms) sin intervención de aprovisionamiento.

Seguridad Inyectada: Identidad subyacente sin confianza en el cliente.

Contratos Rígidos: Prevención de fallos en producción mediante tipado restrictivo pre-compilación.

Escalabilidad Pasiva: Arquitectura que escala granularmente basándose puramente en demanda HTTP, manteniendo un mantenimiento operativo igual a cero.

Conclusiones del Proyecto

Validación Serverless

La arquitectura basada en eventos (Lambda + API Gateway) demostró ser el enfoque ideal para microblogging, manejando ráfagas de tráfico sin requerir aprovisionamiento manual de servidores.

Complejidad Controlada

El uso de DynamoDB en modo On-Demand, apoyado por índices secundarios globales (GSIs), resolvió el problema del timeline y consultas complejas manteniendo baja latencia operativa.

Seguridad Inherente

Delegar la autenticación a Cognito e inyectar el identificador seguro (sub) desde el backend cerró brechas de seguridad comunes, demostrando madurez en el control de acceso.